- we hover a dot
- our `onVoronoiMouseEnter()` function will be triggered
- our hover lines will show
- our hover lines will capture the mouse
- our `onVoronoiMouseLeave()` function will be triggered, since our pointer is now hovering a hover line
- our hover lines will be removed
- we'll hover a dot, now that our hover lines are gone
- etc

To prevent the hover lines from capturing the hover event, we can add the `pointer-events: none` CSS property to our hover lines. This will prevent them from capturing any mouse events.

We'll also smooth the movement of our hover lines with a CSS transition and decrease their opacity.

```
pointer-events: none;
transition: all 0.2s ease-out;
opacity: 0.5;
```

Scatter plot with lightened hover lines

Our transitions are smoother and this reduces the prominence of our bars somewhat, but they're still very "loud". Here's a fun trick: we can use a blend mode to change how the color interacts with its "background".

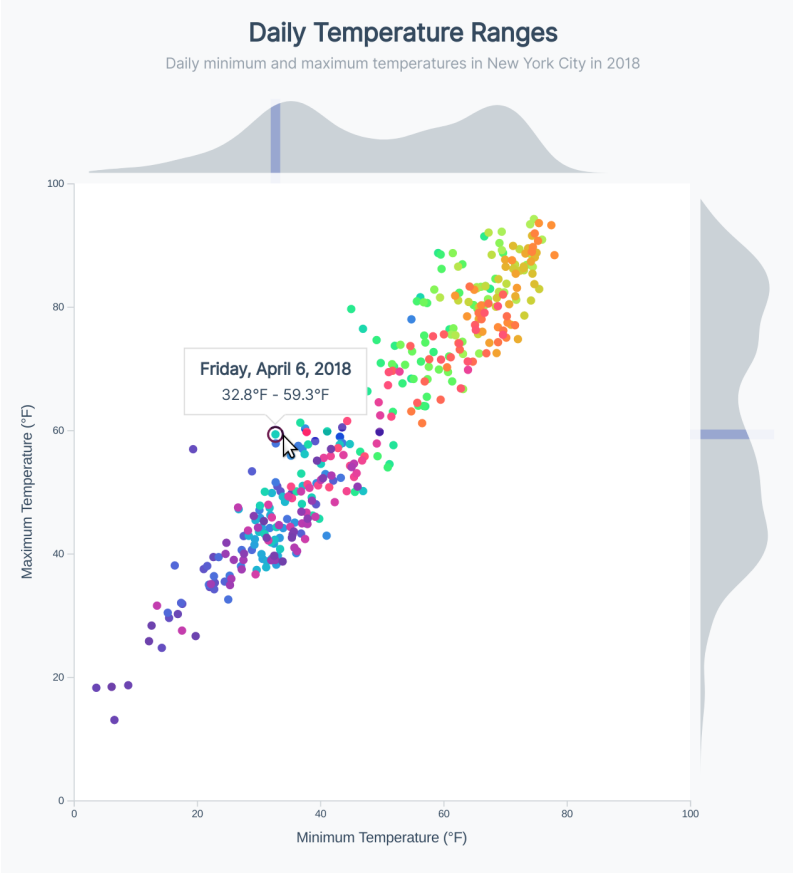
We can set the blend mode by using the `mix-blend-mode` CSS property.

```
mix-blend-mode: color-burn;
```

There are many blend mode options — play around with a few of them! For example, `color-dodge`, `overlay`, `hard-light`, and `difference`. I'll stick with `color-burn` here, since it makes our line almost invisible outside of the histograms, which is

great because we don't want it obstructing other elements in our **bounds**. This way, it shows where the values lie in our histograms, but doesn't get in the reader's way.

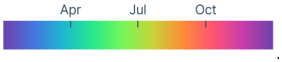
> Read more about `mix-blend-mode` in the MDN docs[a].
>
> [a] https://developer.mozilla.org/en-US/docs/Web/CSS/mix-blend-mode



Scatter plot with blended hover lines

Adding a legend

The meaning of each color isn't clear to our readers — let's add a legend in the bottom, right of our **bounds**. We want this legend to be a bar containing our gradient, with a few dates called out on top.



Scatter plot gradient

To start, we'll add our legend bar's dimensions to our `dimensions` object.

code/10-marginal-histogram/completed/scatter.js

```
20  let dimensions = {
21    width: width,
22    height: width,
23    margin: {
24      top: 90,
25      right: 90,
26      bottom: 50,
27      left: 50,
28    },
29    histogramMargin: 10,
30    histogramHeight: 70,
31    legendWidth: 250,
32    legendHeight: 26,
33  }
```

Next, we'll create a new `<g>` element to position our legend. Let's place our legend in the bottom, right of our chart, since there won't be any days in our dataset with a lower maximum temperature than minimum temperature (and therefore, no dots in the lower, right-hand corner of our chart).

This code can go at the end of our **Draw peripherals** step, after we've created our axes.

code/10-marginal-histogram/completed/scatter.js

```
183   const legendGroup = bounds.append("g")
184       .attr("transform", `translate(${
185         dimensions.boundedWidth - dimensions.legendWidth - 9
186       },${
187         dimensions.boundedHeight - 37
188       })`)
```

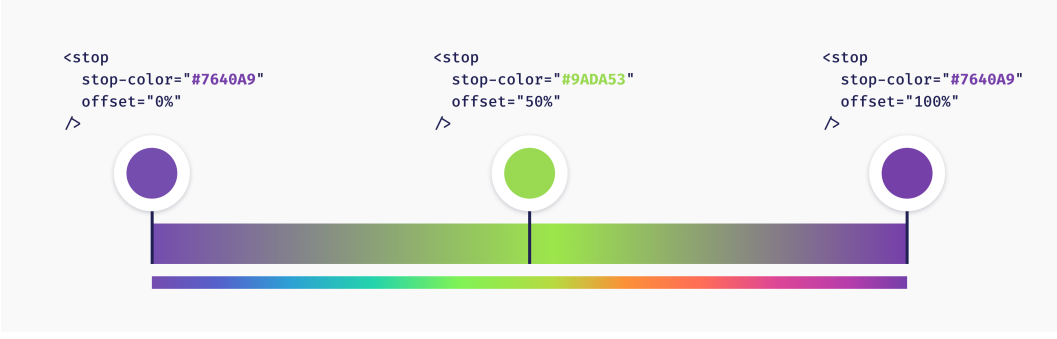
Next, we need to create a gradient. This will be similar to the gradient we created in **Chapter 6** for our map, but with more `<stops>`.

We'll create our gradient within a `<defs>` element, to keep our code organized so we know where to find re-useable elements.

code/10-marginal-histogram/completed/scatter.js

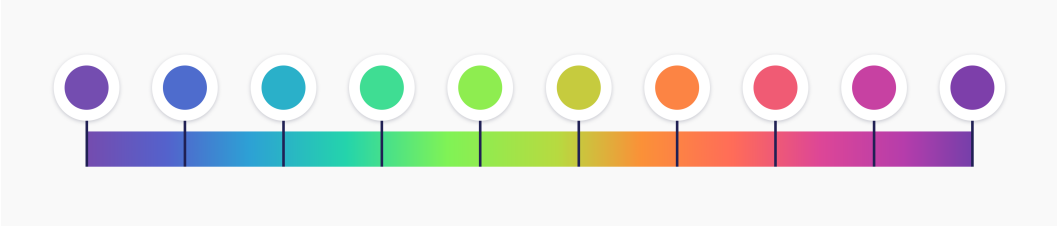
```
190   const defs = wrapper.append("defs")
```

To create a gradient that will match our color scale, we'll want to create an array of equally-spaced colors within our gradient. We'll want a good number of stops — if we only have 3 stops, our gradient will end up looking like this.



Gradient with 3 stops

Instead, let's pick 10 colors within our color scale, so our gradient will better represent our colors.



Gradient with 10 stops

To pick those colors, we want an array of 10 numbers spanning our `colorScale`'s domain (`0` to `1`). We can use `d3.range(n)` to create an array of `n` elements. For example,

```
d3.range(5)
```

will create the array `[0, 1, 2, 3, 4]`. Let's create an array of the number of stops we want (`10`) and normalize the indices to count up to `1`.

code/10-marginal-histogram/completed/scatter.js

```
192   const numberOfGradientStops = 10
193   const stops = d3.range(numberOfGradientStops).map(i => (
194     i / (numberOfGradientStops - 1)
195   ))
```

Now we can use our `stops` array to create one `<stop>` per item, with the matching color and offset percent.

> Note that we want to set the `id` of our gradient so we can reference it later.

code/10-marginal-histogram/completed/scatter.js

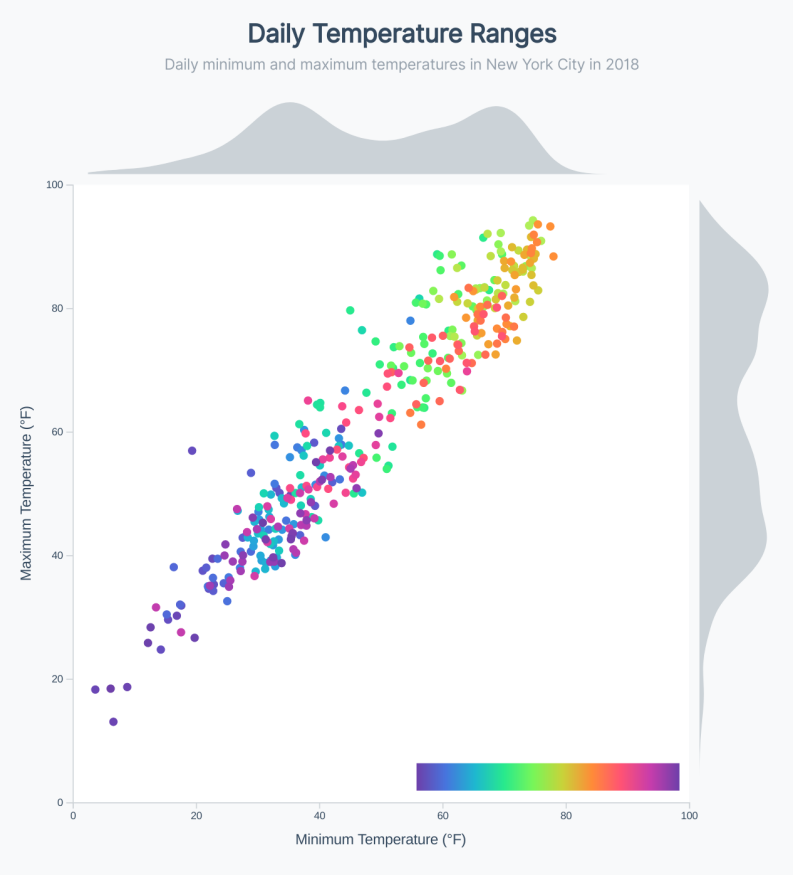
```
196   const legendGradientId = "legend-gradient"
197   const gradient = defs.append("linearGradient")
198       .attr("id", legendGradientId)
199     .selectAll("stop")
200     .data(stops)
201     .enter().append("stop")
202       .attr("stop-color", d => d3.interpolateRainbow(-d))
203       .attr("offset", d => `${d * 100}%`)
```

Now we can create a `<rect>` to display our gradient, using the same `id` that we set for it.

code/10-marginal-histogram/completed/scatter.js

```
205   const legendGradient = legendGroup.append("rect")
206       .attr("height", dimensions.legendHeight)
207       .attr("width", dimensions.legendWidth)
208       .style("fill", `url(#${legendGradientId})`)
```

Looking good! If you're following along, try playing with different numbers of stops to see how the granularity of the gradient affects its appearance.



Scatter plot with gradient bar

This gradient isn't much help by itself — let's add ticks, similar to our chart axes.We could use one of d3's axis generators (eg `d3.axisBottom()`), but that seems like overkill here.

We want to label the start of a few key months within the year — because we want to be so specific, let's hard-code an array of a few key dates.

code/10-marginal-histogram/completed/scatter.js

```
210   const tickValues = [
211     d3.timeParse("%m/%d/%Y")(`4/1/${colorScaleYear}`),
212     d3.timeParse("%m/%d/%Y")(`7/1/${colorScaleYear}`),
213     d3.timeParse("%m/%d/%Y")(`10/1/${colorScaleYear}`),
214   ]
```

Next, we'll create a scale to help position our tick `<text>` elements — this scale should convert a date into an x-position on our legend.

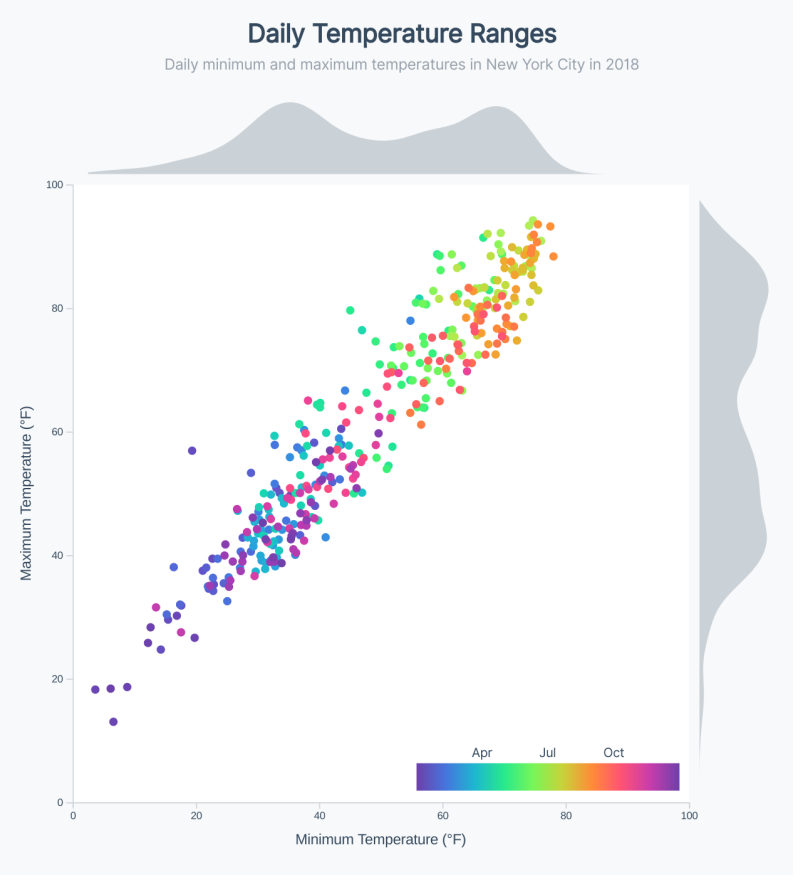
code/10-marginal-histogram/completed/scatter.js

```
215   const legendTickScale = d3.scaleLinear()
216       .domain(colorScale.domain())
217       .range([0, dimensions.legendWidth])
```

Now we can use our `tickValues` array to create `<text>` elements along our legend gradient.

code/10-marginal-histogram/completed/scatter.js

```
219   const legendValues = legendGroup.selectAll(".legend-value")
220     .data(tickValues)
221     .enter().append("text")
222       .attr("class", "legend-value")
223       .attr("x", legendTickScale)
224       .attr("y", -6)
225       .text(d3.timeFormat("%b"))
```



Scatter plot with gradient ticks

We'll also add tick lines, since it's not very clear *where* **March 2nd** is on the gradient.

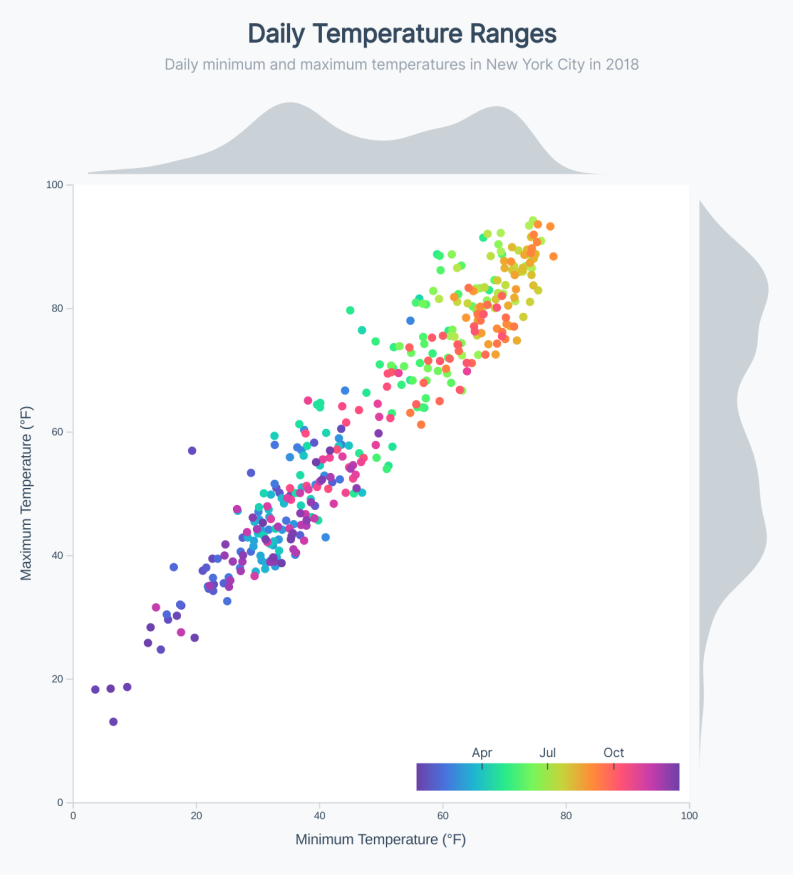
code/10-marginal-histogram/completed/scatter.js

```
227   const legendValueTicks = legendGroup.selectAll(".legend-tick")
228     .data(tickValues)
229     .enter().append("line")
230       .attr("class", "legend-tick")
231       .attr("x1", legendTickScale)
232       .attr("x2", legendTickScale)
233       .attr("y1", 6)
```

These lines won't show up just yet — we need to add a `stroke` color in our `styles.css` file. While we're in there, let's also bump down the font size of our ticks and make sure they're horizontally centered with the tick.

```
.legend-value {
    font-size: 0.76em;
    text-anchor: middle;
}

.legend-tick {
    stroke: #34495e;
}
```

Wonderful! Now our readers will have a better idea of what time of year each color corresponds to.

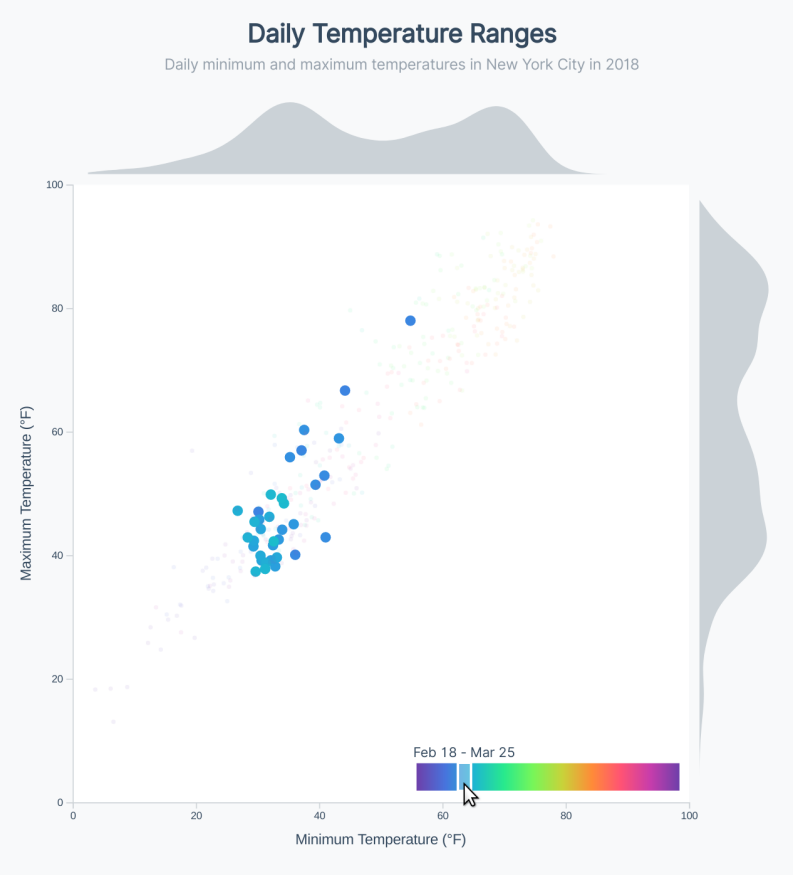


Scatter plot with legend tick marks

Highlight dots when we hover the legend

At last, we come to the most exciting part of this chart. One of the best parts of creating charts in d3 and Javascript is that *we can make anything we think of.* This includes fun interactions that invite the reader to explore the data themselves.

Let's highlight dots in a similar date range when a user moves their mouse over the legend.



Scatter plot legend hover

Right now, nothing happens when we hover the legend. Let's add mouse events to capture mouse events at the end of our **Set up interactions** step. Since we want to do something every time the reader's mouse *moves*, we'll listen for `"mousemove"` events instead of `"mouseenter"` events.

```
legendGradient.on("mousemove", onLegendMouseMove)
  .on("mouseleave", onLegendMouseLeave)

function onLegendMouseMove(e) {
}
function onLegendMouseLeave() {
}
```

Next, we'll create our static hover elements *right before we define our* `onLegendMouseMove()` *function*. The reason we want to do this outside of our function is to prevent from creating new elements every time we move our mouse over the legend. It's more performant and cleaner code.

We'll set the width of the "selected" region of our legend, then create a `<g>` to house our hover elements and hide it for now. This group will serve a similar purpose to the `hoverElementsGroup` we created before - we'll only have to show and hide one element in our mouse events, as opposed to having to remember every single element that is visible on hover.

code/10-marginal-histogram/completed/scatter.js

```
335   const legendHighlightBarWidth = dimensions.legendWidth * 0.05
336   const legendHighlightGroup = legendGroup.append("g")
337       .attr("opacity", 0)
```

Now we can create our hover elements - a "bar" that will show what region of the legend we're highlighting and a `<text>` element to show the dates.

code/10-marginal-histogram/completed/scatter.js

```
338   const legendHighlightBar = legendHighlightGroup.append("rect")
339       .attr("class", "legend-highlight-bar")
340       .attr("width", legendHighlightBarWidth)
341       .attr("height", dimensions.legendHeight)
342
343   const legendHighlightText = legendHighlightGroup.append("text")
344       .attr("class", "legend-highlight-text")
345       .attr("x", legendHighlightBarWidth / 2)
346       .attr("y", -6)
```

> Note that we're shifting our text over by half of the bar width. We need to do this in order to center the text with the bar — the group will expand to be as wide as our bar and when we use the `text-anchor: middle` CSS property, we want it to be aligned with the *center of the bar*, instead of the left side of the bar.

Perfect! Now let's flesh out our `onLegendMouseMove()` function. First, we'll need to figure out *what dates we're hovering*. To find the point we're hovering, we need to find our mouse's x position relative to our legend.

We can use the `d3.mouse()` function to get back an `[x, y]` coordinates. These coordinates will be relative to the element that we pass to `d3.mouse()` — we can give it `this`, which is the element that we initialized our mouse event listener on (`legendGradient`).

```
function onLegendMouseMove(e) {
  const [x] = d3.mouse(this)

}
```

> We're using ES6 array destructuring to grab the `x` value directly — this is the equivalent of: {lang=javascript,line-numbers=off}
> const coordinates = d3.mouse(this) const x = coordinates[0]

Now that we have the x position of our mouse on the legend, let's calculate the minimum and maximum range of dates that we we're going to highlight. Remember that we can use a scale's `.invert()` method to convert values from the `range` dimension to the `domain` dimension. If we use the `legendTickScale` we used previously to position our legend tick values, we'll be able to convert a legend x position into a date.

code/10-marginal-histogram/completed/scatter.js

```
353     const minDateToHighlight = new Date(
354       legendTickScale.invert(x - legendHighlightBarWidth)
355     )
356     const maxDateToHighlight = new Date(
357       legendTickScale.invert(x + legendHighlightBarWidth)
358     )
```

If we use the x position of our mouse to position our highlighted bar, it will go out-of-bounds when we approach the left or right side. An easy way to bound a value is to use `d3.median()` which will sort the values in a passed array and pick the middle value. This works to our favor — when we pass an array of `[min, value, max]`, there are three possible scenarios:

- the `value` is lower than the `min`, resulting in the sorted array `[value, min, max]`, making `min` the middle value
- the `value` is between the `min` and the `max`, resulting in the sorted array `[min, value, max]`, making `min` the middle value
- the `value` is higher than the `max`, resulting in the sorted array `[min, max, value]`, making `max` the middle value

Let's use that to bound our bar's x position by `0` (the beginning of our legend) and the legend's width minus the highlight bar's width.

code/10-marginal-histogram/completed/scatter.js

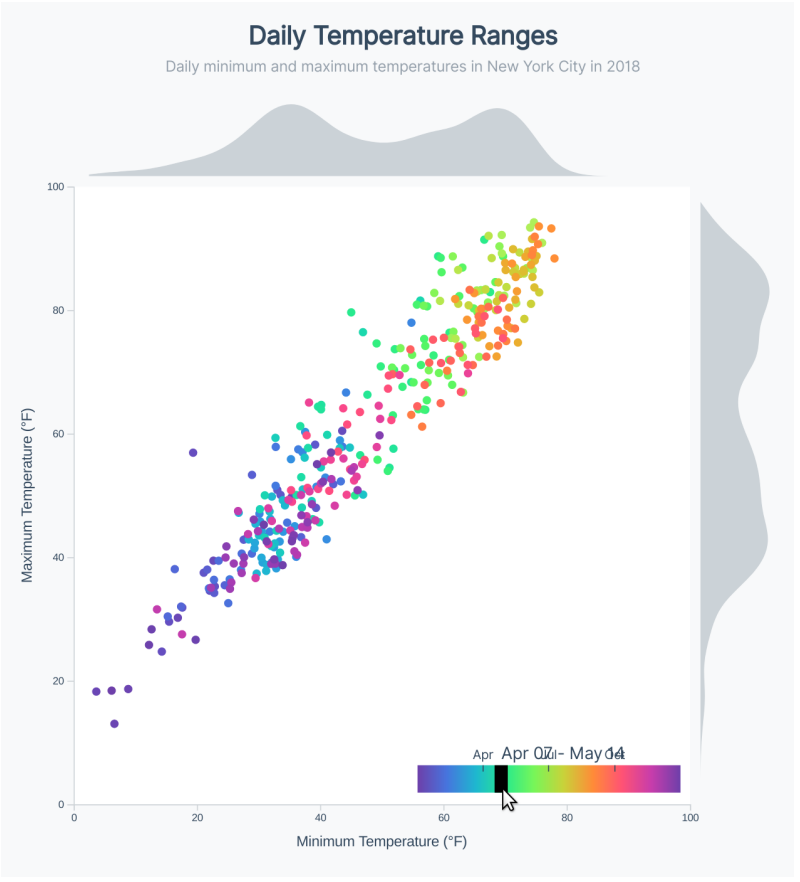
```
360     const barX = d3.median([
361       0,
362       x - legendHighlightBarWidth / 2,
363       dimensions.legendWidth - legendHighlightBarWidth,
364     ])
```

Lastly, let's show and position our `legendHighlightGroup` and update the text of our `legendHighlightText`. We want it to display the minimum and maximum dates, separated by a hyphen.

```
legendHighlightGroup.style("opacity", 1)
    .style("transform", `translateX(${barX}px)`)
legendHighlightText.text([
    d3.timeFormat("%b %d")(minDateToHighlight),
    d3.timeFormat("%b %d")(maxDateToHighlight),
].join(" - "))
```

> Notice that we're using the padded version of the date's day: `"%-d"` will format March 2nd as `2` whereas `"%d"` will zero-pad the day and format it as `02`. While a little less readable, this will keep the date from jumping around when it switches from a 1-digit day to a 2-digit day. Try both versions to see the difference for yourself!

Okay great, now we can see where on the legend we're hovering, and the date range.

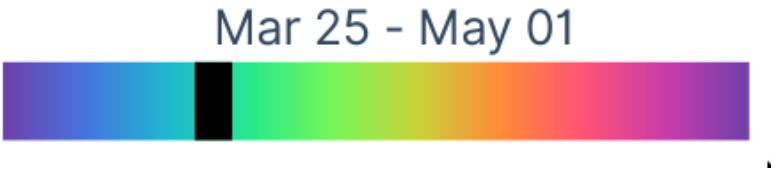


Scatter plot legend on hover

It's a bit hard to read, though. Let's also dim the normal legend ticks so they don't distract from the highlighted portion.

```
legendValues.style("opacity", 0)
legendValueTicks.style("opacity", 0)
```

Although usually removing information is a bad idea, it works here because it's initiated by user input, and the user can easily get it back.

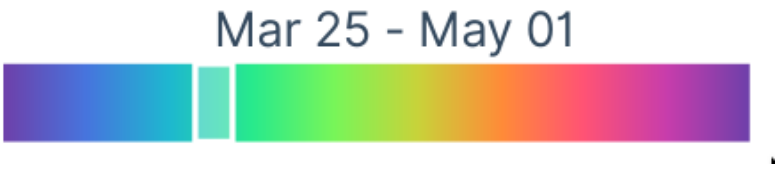


Scatter plot legend on hover

Much cleaner! An additional issue is that we're obscuring the very colors we want to highlight - let's add some styles to our highlight bar to show the covered colors. We'll also want to prevent our bar from capturing mouse events, to keep it from blocking mouse movement on our legend bar.

```
.legend-highlight-bar {
    fill: rgba(255, 255, 255, 0.3);
    stroke: white;
    stroke-width: 2px;
    pointer-events: none;
}
```

That's looking much clearer!

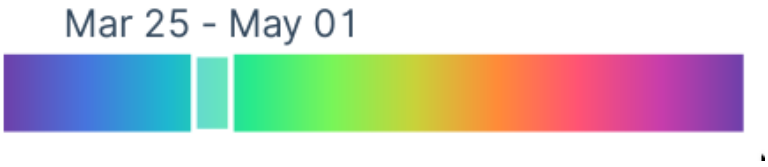


Scatter plot legend on hover

Let's center our text, shrink the size, and make each character the same width (to prevent the letters from jumping around as we hover months with characters of different widths).

```
.legend-highlight-text {
    text-anchor: middle;
    font-size: 0.8em;
    font-feature-settings: 'tnum' 1;
}
```

That feels much better when we move our mouse around the legend bar.

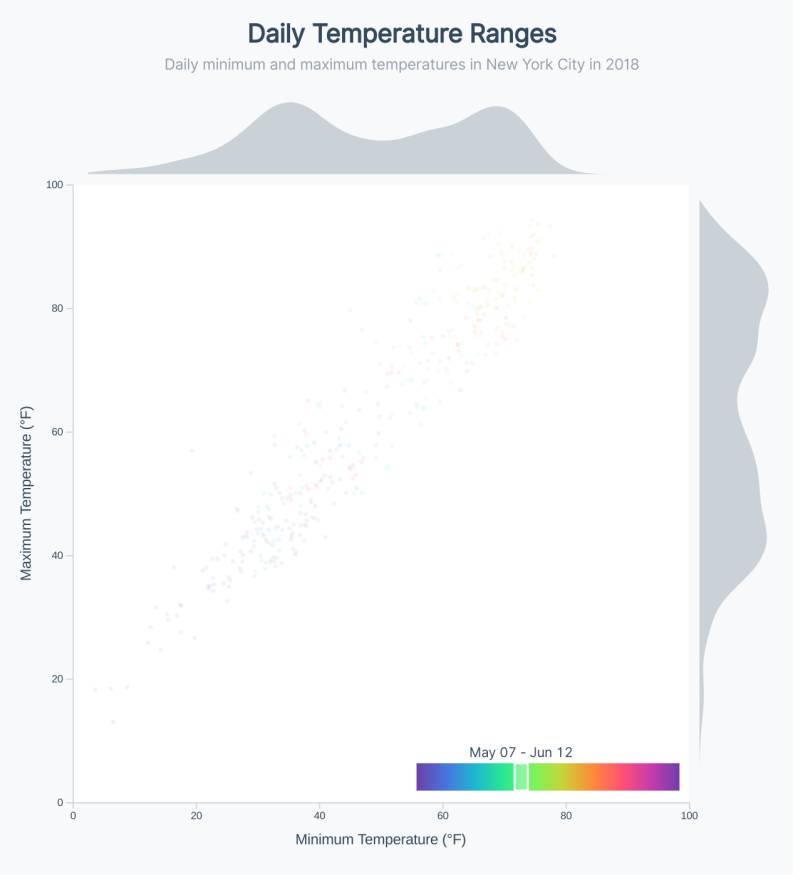


Scatter plot legend on hover

Now let's update our scatter plot! First, we'll want to *dim and shrink all of our dots* — let's animate this change, but keep the duration short to keep from making the interaction feel laggy.

```
dots.transition().duration(100)
    .style("opacity", 0.08)
    .attr("r", 2)
```

Now all of our dots almost disappear when we hover our legend, perfect!



Scatter plot legend on hover

Next, we want to update the days within our highlighted range. First, we'll create an `isDayWithinRange()` function that converts a single data point into a date and returns whether or not it's in-between our min and max highlighted dates.

There are two special edge cases we want to handle — when we hover a date near the left edge, we want to highlight nearby dates on the right edge, and vice versa.

code/10-marginal-histogram/completed/scatter.js

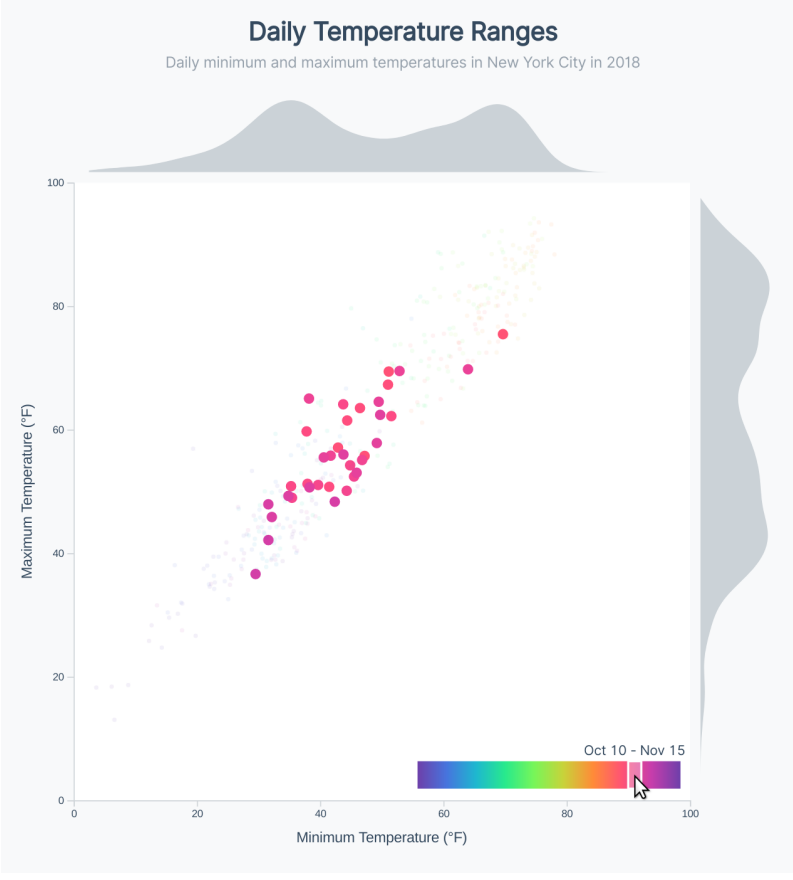
```
    const getYear = d => +d3.timeFormat("%Y")(d)
    const isDayWithinRange = d => {
      const date = colorAccessor(d)

      if (getYear(minDateToHighlight) < colorScaleYear) {
        // if dates wrap around to previous year,
        // check if this date is after the min date
        return date >= new Date(minDateToHighlight)
          .setYear(colorScaleYear)
          || date <= maxDateToHighlight

      } else if (getYear(maxDateToHighlight) > colorScaleYear) {
        // if dates wrap around to next year,
        // check if this date is before the max date
        return date <= new Date(maxDateToHighlight)
          .setYear(colorScaleYear)
          || date >= minDateToHighlight

      } else {
        return date >= minDateToHighlight
          && date <= maxDateToHighlight
      }
    }
```

Now we can pass our `isDayWithinRange()` function to a d3 selection object's `.filter()` method to select only those dots that fall within our highlighted date range. We'll transition those dots back to full opacity and a slightly-larger-than-normal radius.

code/10-marginal-histogram/completed/scatter.js

```
405     const relevantDots = dots.filter(isDayWithinRange)
406       .transition().duration(100)
407         .style("opacity", 1)
408         .attr("r", 5)
```

Now when we move our mouse over the legend, we can see that we're highlighting a groups of dots that corresponds to our hover position.



Scatter plot legend with highlighted dots

Our dots stay highlighted, even when we move our mouse away from our legend. Let's update our empty `onLegendMouseLeave()` function to fade all of our dots back

in and reset our legend.

```
function onLegendMouseLeave() {
  dots.transition().duration(500)
      .style("opacity", 1)
      .attr("r", 4)

  legendValues.style("opacity", 1)
  legendValueTicks.style("opacity", 1)
  legendHighlightGroup.style("opacity", 0)
}
```

Mini hover histograms

This chart is fantastic already, but let's take it *one step* further. Since we already have marginal histograms, let's overlay a histogram of the highlighted points to show how their distribution compares to the overall distribution of temperatures.

This sounds really complicated, but will be fairly simple, since we're building on the awesome code we've already written.

To start, we'll create our static hover elements (this code can go right before our `onLegendMouseMove()` function). We'll want one `<path>` in our top histogram **bounds** and another `<path>` in our right histogram **bounds**. This way, the highlight histograms will be positioned in the same place as our main histograms.

code/10-marginal-histogram/completed/scatter.js

```
348   const hoverTopHistogram = topHistogramBounds.append("path")
349   const hoverRightHistogram = rightHistogramBounds.append("path")
```

> Note that we don't need to hide these `<path>`s off the bat because they won't be visible without a `d` attribute string.

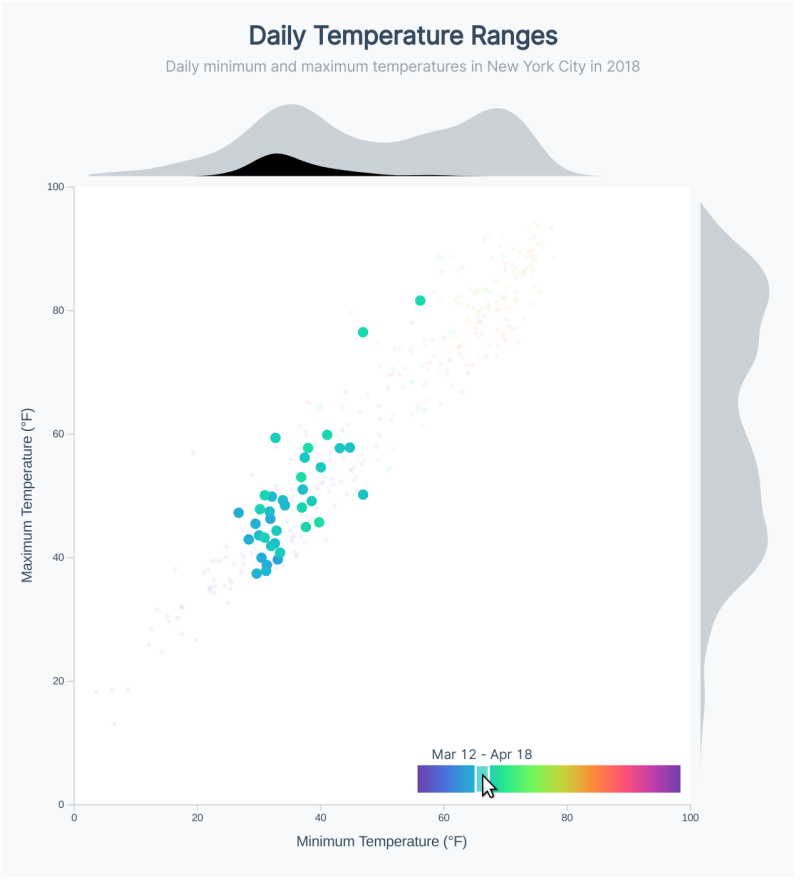
Now let's add code to update these static elements right before the end of our `onLegendMouseMove()` function.

We want to create a histogram data structure with the histogram generator we created for our main top histogram (`topHistogramGenerator()`) and pass that to our histogram line generator `topHistogramLineGenerator()` to create our `<path>`'s `d` attribute string.

That's a complicated step — let's see how it would look in code:

```
const hoveredDate = d3.isoParse(legendTickScale.invert(x))
const hoveredDates = dataset.filter(isDayWithinRange)

hoverTopHistogram.attr("d", d => (
  topHistogramLineGenerator(topHistogramGenerator(hoveredDates))
))
```

Boom! Now we can see a smaller dark histogram overlaid over our top histogram, and it updates when we move our mouse over the legend.

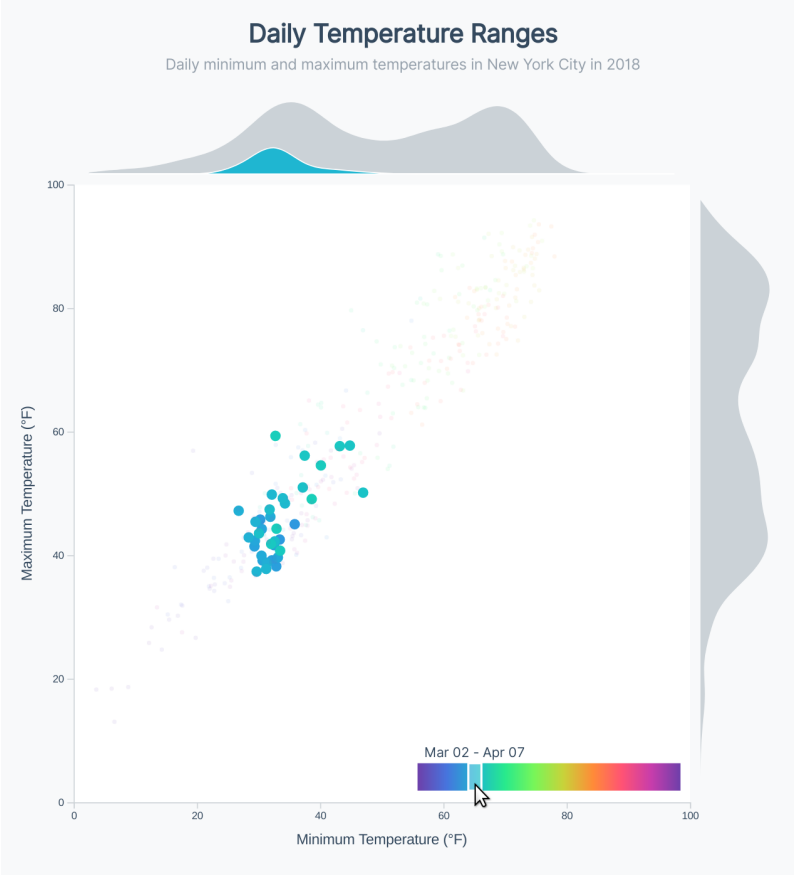


Scatter plot legend with top highlighted histogram

Let's update the styles a bit — we'll set the `fill` to use our hovered date's color to reinforce the relationship between the legend, the highlighted dots, and the mini histogram. We'll also add a white stroke (to m ake the distinction between the two histograms a bit more clear), and make our path visible.

```
.attr("fill", colorScale(hoveredDate))
.attr("stroke", "white")
.style("opacity", 1)
```

Looking great! It's wonderful how much we can do with a few lines of code, once most of our chart is already set up.

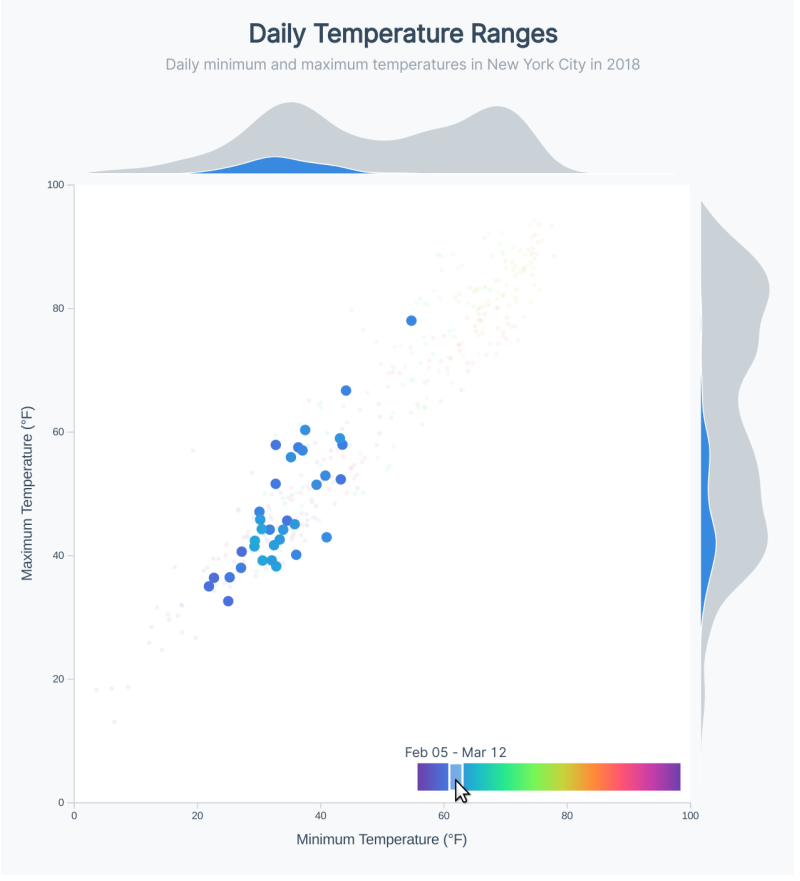


Scatter plot legend with top highlighted histogram

Let's give our right highlighted histogram the same treatment.

```
hoverRightHistogram.attr("d", d => (
      rightHistogramLineGenerator(rightHistogramGenerator(hoveredDates))
    ))
    .attr("fill", colorScale(hoveredDate))
    .attr("stroke", "white")
    .style("opacity", 1)
```

Now we can see both histograms update when we move over our legend.



Scatter plot legend with both highlighted histograms

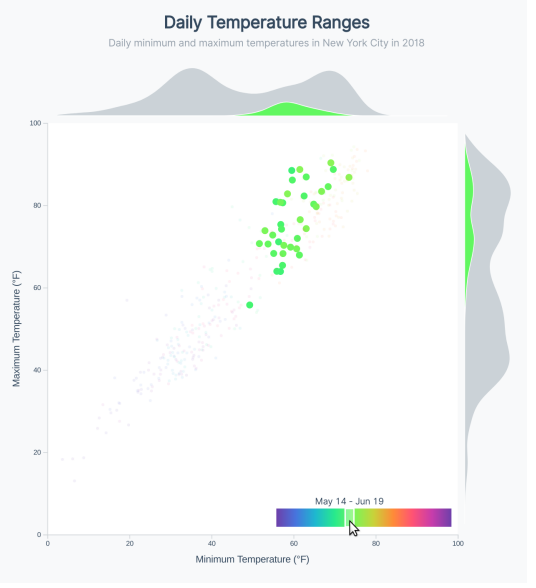
Our last step is to hide our histograms when our mouse leaves the legend. We'll set the `opacity` of our highlight histograms at the end of our `onLegendMouseLeave()` function.

```
hoverTopHistogram.style("opacity", 0)
hoverRightHistogram.style("opacity", 0)
```

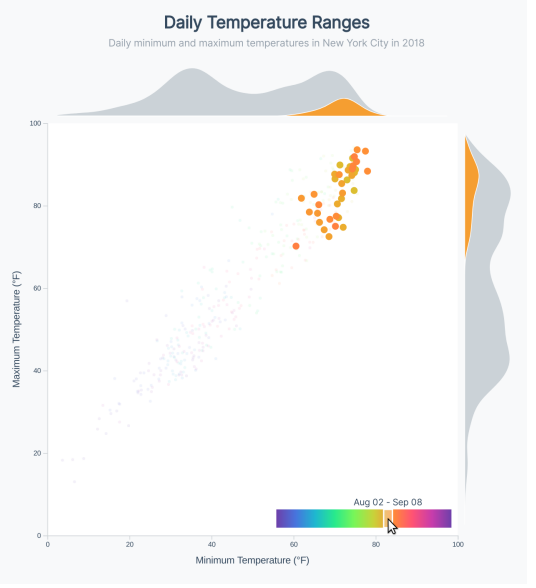
And we're finished! Take a minute to play around with the chart and bask in the glory of your great accomplishment! This chart was no easy feat, but you powered through.

Hopefully you can get a sense of how the interactions add to the effectiveness of this

chart. There are many insights to be found from exploring this chart. For example, we can see that mid-May to mid-June has some of the hottest temperatures, but the days' minimum temperatures aren't so hot. Most of the days with the highest minimum temperature are actually in August.



Finished scatter plot

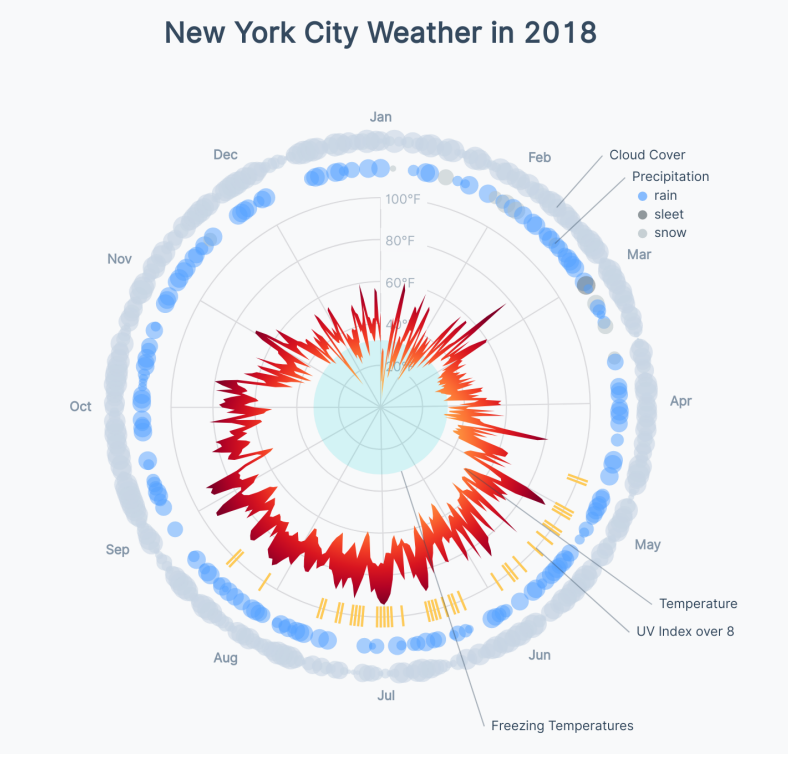


Finished scatter plot

If you have your own data, look around for interesting insights and share them to show off your handiwork!

Radar Weather Chart

We talked about **radar charts** in **Chapter 8**. For this project, we'll build a more complex radar chart.



Radar Weather Chart

This chart will give the viewer a sense of overall weather for the whole year, and will highlight trends such as:

- What time of the year is cloudiest, and does that correlate with the coldest days?
- Do cloudy days have lower UV indices, or are they rainier?
- How does weather vary per season?

The complexity of this chart will increase the amount of time the viewer needs to wrap their head around it. But once the viewer understands it, they can answer more nuanced questions than a simpler chart.

We'll reinforce concepts we've already learned as well as new concepts, such as angular math, as we build this visualization. Let's get started!

Getting set up

Once your server is running (`live-server`), find the page we'll be working on at http://localhost:8080/11-radar-weather-chart/draft/[66], and open the code folder at `/code/11-radar-weather-chart/draft/`. Feel free to update the chart title in `index.html` if you're using your own data.

[66] http://localhost:8080/11-radar-weather-chart/draft/

Start

We'll run through this example fairly quickly, so if you need a reference, the completed chart is in the `/code/11-radar-weather-chart/completed/` folder.

Accessing the data

To start, let's create our data accessors. Doing this up front will remind us of the structure of our data, and let us focus on drawing our chart later.

Let's look at the first data point by logging it out to our console after we fetch our data:

```
const dataset = await d3.json("./../../../my_weather_data.json")
console.table(dataset[0])
```

chart.js:6

(index)	Value
time	1514782800
summary	"Clear throughout the day."
icon	"clear-day"
sunriseTime	1514809280
sunsetTime	1514842810
moonPhase	0.48
precipIntensity	0
precipIntensityMax	0
precipProbability	0
temperatureHigh	18.39
temperatureHighTime	1514836800
temperatureLow	12.23
temperatureLowTime	1514894400
apparentTemperatureHigh	17.29
apparentTemperatureHighTime	1514844000
apparentTemperatureLow	4.51
apparentTemperatureLowTime	1514887200
dewPoint	-1.67
humidity	0.54
pressure	1028.26
windSpeed	4.16
windGust	13.98
windGustTime	1514829600
windBearing	309
cloudCover	0.02
uvIndex	2
uvIndexTime	1514822400
visibility	10
temperatureMin	6.17
temperatureMinTime	1514808000
temperatureMax	18.39
temperatureMaxTime	1514836800
apparentTemperatureMin	-2.19
apparentTemperatureMinTime	1514808000
apparentTemperatureMax	17.29
apparentTemperatureMaxTime	1514844000
date	"2018-01-01"

▸ Object

Our dataset

Since we already know what our final chart will look like, we can pull out all of the metrics we'll need. Let's create an accessor for each of the metrics we'll plot (*min temperature*, *max temperature*, *precipitation*, *cloud cover*, *uv*, and *date*).

code/11-radar-weather-chart/completed/chart.js

```
7   const temperatureMinAccessor = d => d.temperatureMin
8   const temperatureMaxAccessor = d => d.temperatureMax
9   const uvAccessor = d => d.uvIndex
10  const precipitationProbabilityAccessor = d => d.precipProbability
11  const precipitationTypeAccessor = d => d.precipType
12  const cloudAccessor = d => d.cloudCover
13  const dateParser = d3.timeParse("%Y-%m-%d")
14  const dateAccessor = d => dateParser(d.date)
```

That's a mouthful! Thankfully, we got that out of the way and won't need to look at the exact structure of our data again.

Creating our scales

Next, we'll want to create scales to convert our weather metrics into physical properties so we know where to draw our data elements.

The location of a data element around the radar chart's center corresponds to its date. Let's create a scale that converts a date into an angle.

code/11-radar-weather-chart/completed/chart.js

```
73  // 4. Create scales
74
75  const angleScale = d3.scaleTime()
76    .domain(d3.extent(dataset, dateAccessor))
77    .range([0, Math.PI * 2]) // this is in radians
```

> Note that we're using **radians**, instead of **degrees**. Angular math is generally easier with **radians**, and we'll want to use `Math.sin()` and `Math.cos()` later, which deals with **radians**. There are **2π** radians in a full circle. If you want to know more about **radians**, the Wikipedia entry is a good source[a].
>
> [a] https://en.wikipedia.org/wiki/Radian

Adding gridlines

To get our feet wet with this angular math, we'll draw our peripherals *before* we draw our data elements. Let's switch those steps in our code.

```
// 6. Draw peripherals


// 5. Draw data
```

If your first thought was "but the checklist!", here's a reminder that our chart drawing checklist is here as a friendly guide, and we can switch the order of steps if we need to.

Drawing the grid lines (peripheral) first is helpful in cases like this where we want our data elements to layer *on top*. If we wanted to keep our steps in order, we could also create a `<g>` element *first* to add our grid lines to *after*.

Creating a group to hold our grid elements is also a good idea to keep our elements organized – let's do that now.

code/11-radar-weather-chart/completed/chart.js

```
118   const peripherals = bounds.append("g")
```

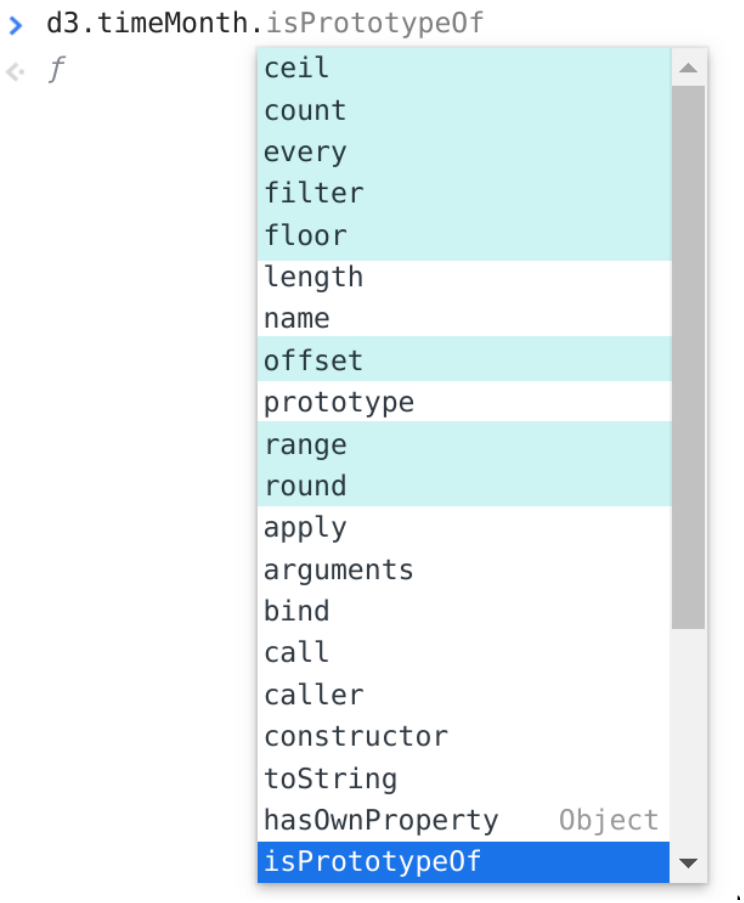
Draw month grid lines

Next, let's create one "spoke" for each month in our dataset. First, we'll need to create an array of each month. We already know what our first and last dates are – they are the `.domain()` of our `angleScale`. But how can we create a list of each month between those two dates?

The **d3-time**[67] module has various *intervals*, which represent various units of time. For example, `d3.timeMinute()` represents every minute and `d3.timeWeek()` represents every week.

Each of these intervals has a few methods – we can see those methods in the documentation, and also as autocomplete options within our dev tools console.

[67] https://github.com/d3/d3-time#intervals

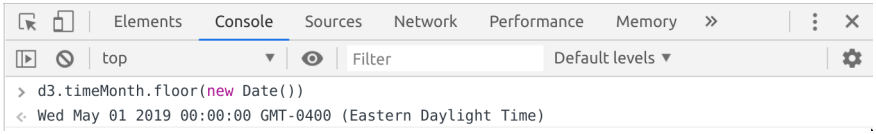


d3.timeMonth() autocomplete options

> Notice that some of the options in this list are for **d3-time** intervals (highlighted in cyan), but most are inherited from prototypical Javascript objects.

For example, we could use the `.floor()` method to get the first "time" in the current month:

```
d3.timeMonth.floor(new Date())
```



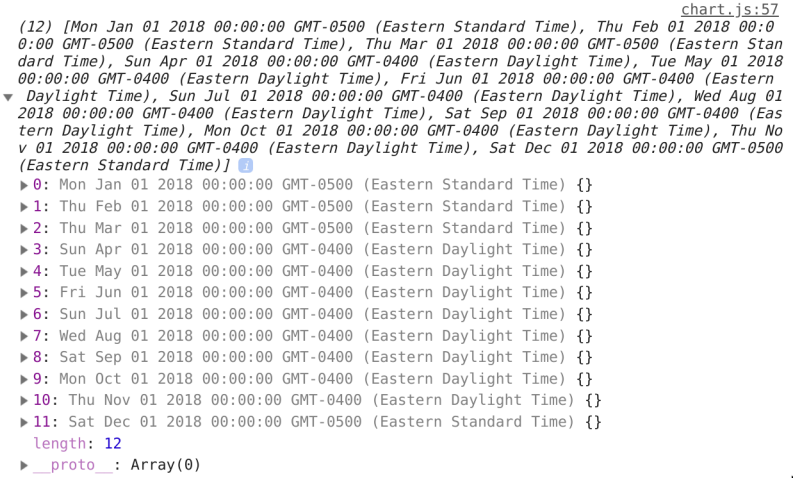
d3.timeMonth.floor() example

d3 time intervals also have a `.range()` method that will return a list of datetime objects, spaced by the specified interval, between two dates, passed as parameters.

Let's try it out by creating our list of months!

```
const months = d3.timeMonth.range(...angleScale.domain())
console.log(months)
```

Great! Now we have an array of datetime objects corresponding to the beginning of each month in our dataset.



months array

d3-time gives us shortcut aliases that can make our code even more concise – we can use `d3.timeMonths()`[68] instead of `d3.timeMonth.range()`.

[68] https://github.com/d3/d3-time#timeMonths

code/11-radar-weather-chart/completed/chart.js

```
120   const months = d3.timeMonths(...angleScale.domain())
```

Let's use our array of months and draw one `<line>` per month.

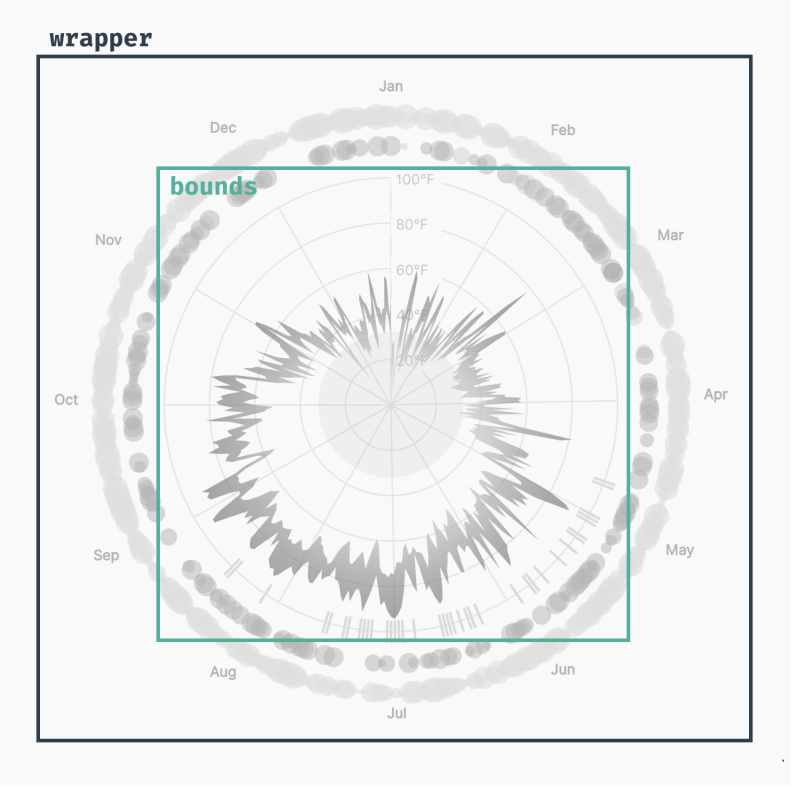
```
const gridLines = months.forEach(month => {
  return peripherals.append("line")
})
```

We'll need to find the angle for each month – let's use our `angleScale` to convert the date into an angle.

```
const gridLines = months.forEach(month => {
  const angle = angleScale(month)

  return peripherals.append("line")
})
```

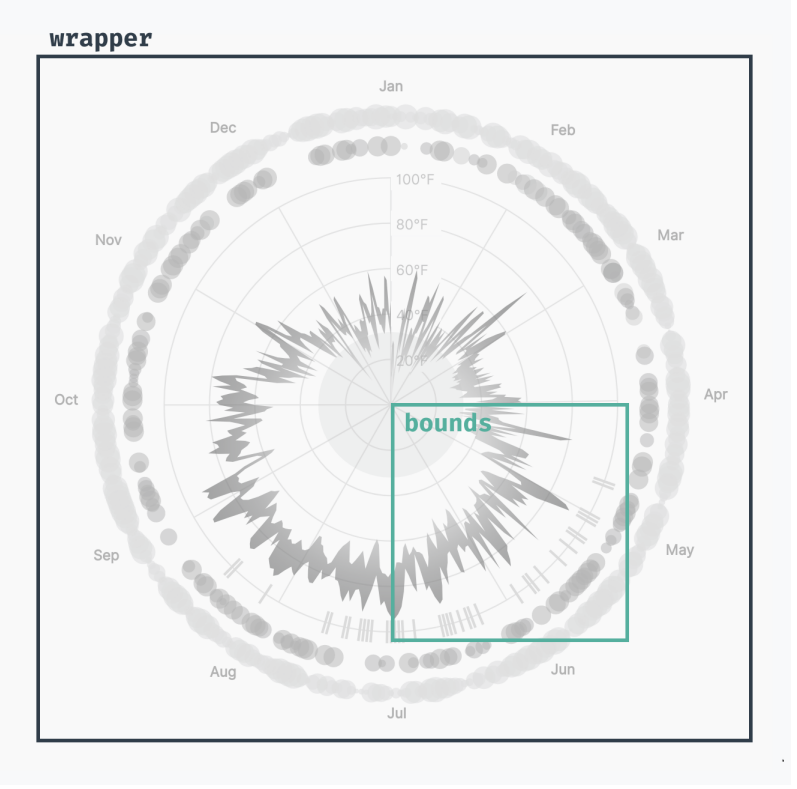
Each *spoke* will start in the middle of our chart – we could start those lines at `[dimensions.boundedRadius, dimensions.boundedRadius]`, but most of our element will need to be shifted in respect to the center of our chart.

Remember how we use our **bounds** to shift our chart according to our `top` and `left` margins?



wrapper & bounds

To make our math simpler, let's instead shift our **bounds** to *start* in the center of our chart.



wrapper & bounds, shifted

This will help us when we decide where to place our data and peripheral elements – we'll only need to know where they lie *in respect to the center of our circle.*

code/11-radar-weather-chart/completed/chart.js

```
51  const bounds = wrapper.append("g")
52      .style("transform", `translate(${
53        dimensions.margin.left + dimensions.boundedRadius
54      }px, ${
55        dimensions.margin.top + dimensions.boundedRadius
56      }px)`)
```

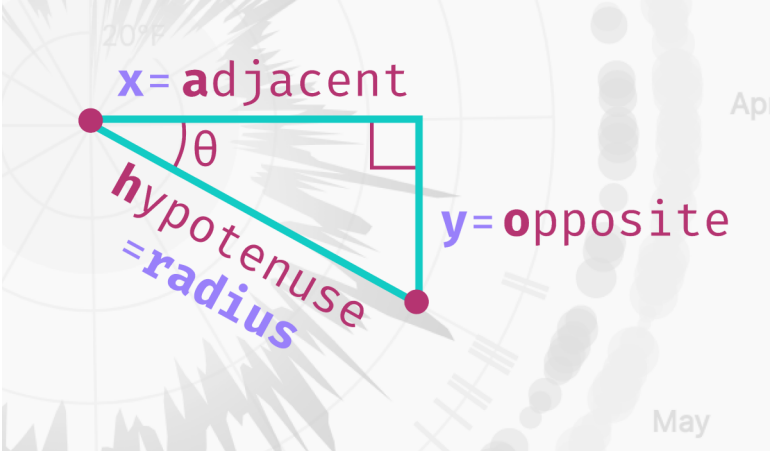
We'll need to convert from *angle* to `[x, y]` coordinate many times in this chart. Let's create a function that makes that conversion for us. Our function will take two parameters:

1. the **angle**
2. the **offset**

and return the `[x,y]` coordinates of a point rotated `angle` radians around the center, and `offset` time our circle's radius (`dimensions.boundedRadius`). This will give us the ability to draw elements at different **radii** (for example, to draw our precipitation bubbles slightly outside of our temperature chart, we'll offset them by `1.14` times our normal radius length).

```
const getCoordinatesForAngle = (angle, offset=1) => []
```

To convert an angle into a coordinate, we'll dig into our knowledge of trigonometry[69]. Let's look at the right-angle triangle (a triangle with a 90-degree angle) created by connecting our *origin point* (`[0,0]`) and our *destination point* (`[x,y]`).



trigonometry triangle parts

The numbers we already know are **theta (θ)** and the **hypotenuse** (`dimensions.boundedRadius * offset`). We can use these numbers to calculate the lengths of the `adjacent` and `opposite` sides of our triangle, which will correspond to the `x` and `y` position of our *destination point*.

[69] https://www.mathsisfun.com/algebra/trigonometry.html

Because our triangle has a *right angle*, we can multiply the `sine` and `cosine` of our `angle` by the length of our **hypotenuse** to calculate our `x` and `y` values (remember the **soh cah toa** mnenomic[70]?).

$$\sin(\theta) = \frac{\text{opposite}}{\text{hypotenuse}}$$

$$\cos(\theta) = \frac{\text{adjacent}}{\text{hypotenuse}}$$

$$\tan(\theta) = \frac{\text{opposite}}{\text{adjacent}}$$

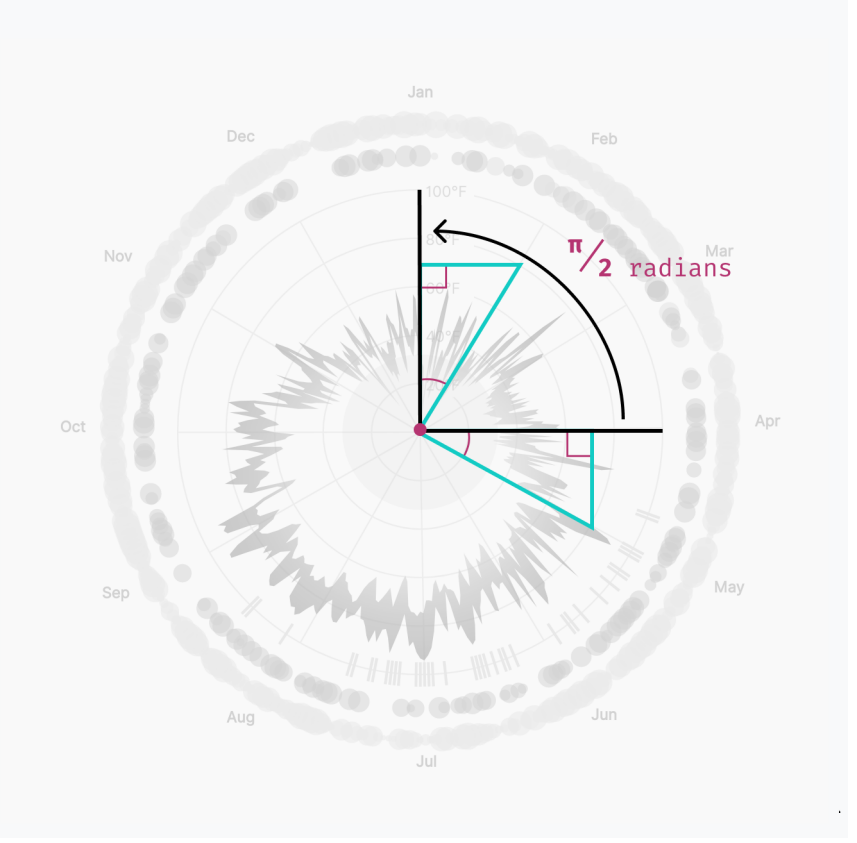
Sohcahtoa

Let's implement this in our `getCoordinatesForAngle()` function.

```
const getCoordinatesForAngle = (angle, offset=1) => [
  Math.cos(angle) * dimensions.boundedRadius * offset,
  Math.sin(angle) * dimensions.boundedRadius * offset,
]
```

This looks great! But we have to make one more tweak to our `getCoordinatesForAngle()` function – an angle of `0` would draw a line horizontally to the right of the *origin point*. But our radar chart starts in the center, above our *origin point*. Let's rotate our angles by 1/4 turn to return the correct points.

[70] http://mathworld.wolfram.com/SOHCAHTOA.html



π / 2 radians rotation

Remember that there are 2π radians in one full circle, so 1/4 turn would be 2π / 4, or π / 2.

code/11-radar-weather-chart/completed/chart.js

```
39   const getCoordinatesForAngle = (angle, offset=1) => [
40     Math.cos(angle - Math.PI / 2) * dimensions.boundedRadius * offset,
41     Math.sin(angle - Math.PI / 2) * dimensions.boundedRadius * offset,
42   ]
```

Whew! That was a a lot of math. Now let's use it to draw our grid lines.

If we move back down in our `chart.js` file, let's grab the `x` and `y` coordinates of the end of our *spokes* and set our `<line>`s' `x2` and `y2` attributes.

> We don't need to set the `x1` or `y1` attributes of our line because they both default to `0`.

```
months.forEach(month => {
 const angle = angleScale(month)
 const [x, y] = getCoordinatesForAngle(angle)

 peripherals.append("line")
   .attr("x2", x)
   .attr("y2", y)
   .attr("class", "grid-line")
})
```

Hmm, we can't see anything yet. Let's give our lines a `stroke` color in our `styles.css` file.

```
.grid-line {
    stroke: #dadadd;
}
```

Finally! We have 12 spokes to show where each of the months in our chart start.

Chart with angle lines

Your spokes might be rotated a bit, depending on when your dataset starts.

Draw month labels

Our viewers won't know which month each spoke is depicting – let's label each of our spokes. While we're looping over our months, let's also get the `[x, y]` coordinates of a point **1.38** times our chart's radius *away* from the center of our chart. This will give us room to draw the rest of our chart within our month labels.

```
months.forEach(month => {
 const angle = angleScale(month)
 const [x, y] = getCoordinatesForAngle(angle)

 peripherals.append("line")
   .attr("x2", x)
   .attr("y2", y)
   .attr("class", "grid-line")

 const [labelX, labelY] = getCoordinatesForAngle(angle, 1.38)
 peripherals.append("text")
   .attr("x", labelX)
   .attr("y", labelY)
   .attr("class", "tick-label")
   .text(d3.timeFormat("%b")(month))
})
```

We can see our month labels now, but there's one issue: the labels on the left are closer to our spokes than the labels on the right.

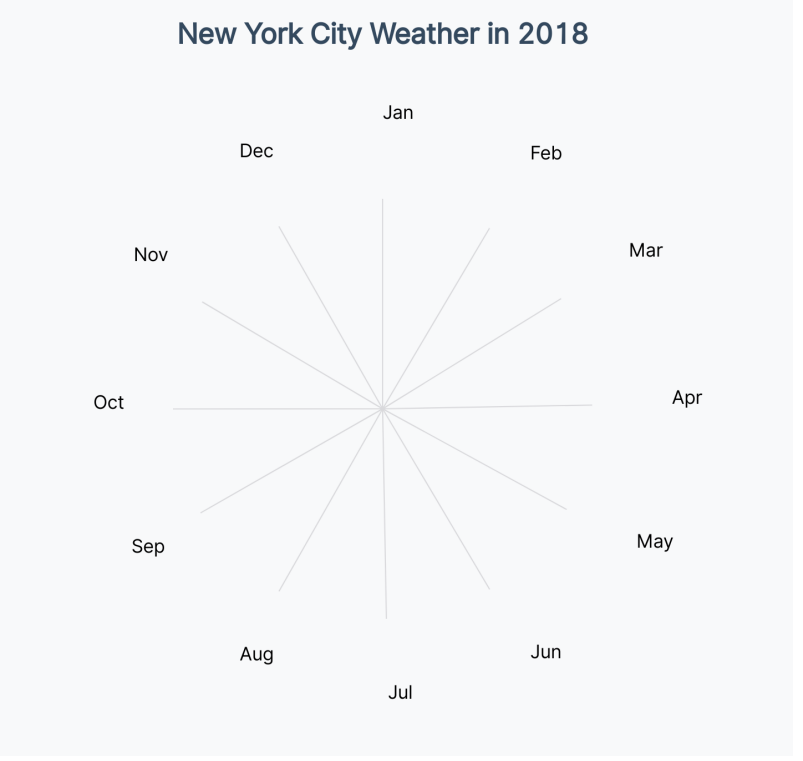


Chart with month labels

This is because our `<text>` elements are anchored by their *left side*. Let's dynamically set their `text-anchor` property, depending on the label's `x` position. We'll align labels on the left by the `end` of the text, and labels near the center by their `middle`.

> Note that `text-anchor` is essentially the `text-align` CSS property for SVG elements.

```
.text(d3.timeFormat("%b")(month))
.style("text-anchor",
  Math.abs(labelX) < 5 ? "middle" :
  labelX > 0           ? "start"  :
                         "end"
)
```

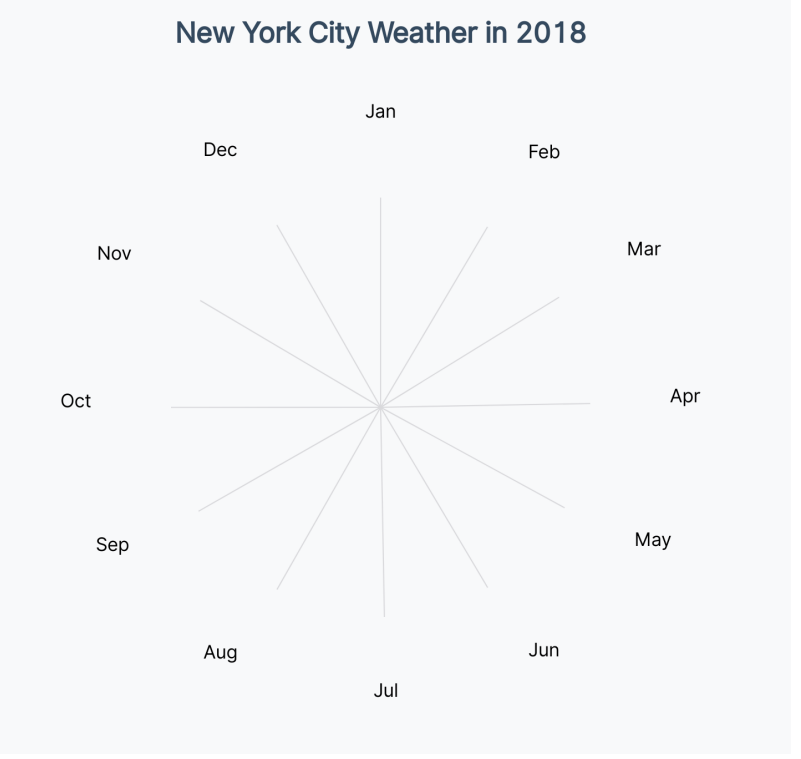


Chart with month labels, anchored property

Our labels also aren't centered vertically with our spokes. Let's center them, using `dominant-baseline`, and update their styling to decrease their visual weight. We want our labels to orient our users, but not to distract from our data.

```
.tick-label {
    dominant-baseline: middle;
    fill: #8395a7;
    font-size: 0.7em;
    font-weight: 900;
    letter-spacing: 0.005em;
}
```

Looking much better!

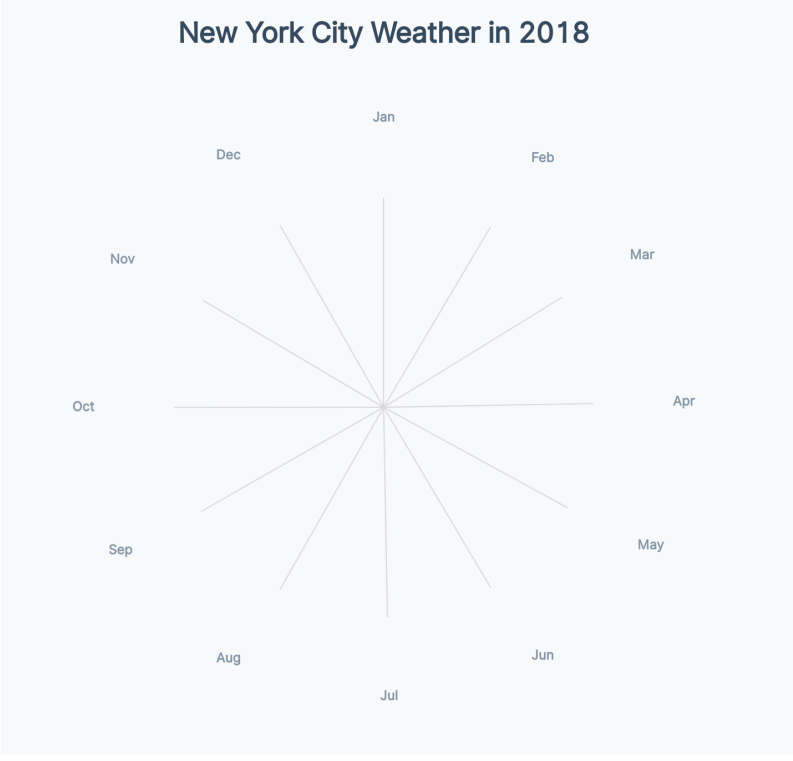


Chart with month labels, styled

Adding temperature grid lines

Our final chart has circular grid marks that mark different temperatures. Before we add this, we need a temperature scale that converts a **temperature** to a **radius**.

Higher temperatures are drawn further from the center of our chart.

Let's add a `radiusScale` at the end of our **Create scales** section. We'll want to use `nice()` to give us friendlier minimum and maximum values, since the exact start and end doesn't matter. Note that we didn't use `.nice()` to round the edges of our `angleScale`, since we want it to start and end exactly with its **range**.

code/11-radar-weather-chart/completed/chart.js

```
79  const radiusScale = d3.scaleLinear()
80    .domain(d3.extent([
81      ...dataset.map(temperatureMaxAccessor),
82      ...dataset.map(temperatureMinAccessor),
83    ]))
84    .range([0, dimensions.boundedRadius])
85    .nice()
```

> We're using the ES6 *spread* operator (`...`) to *spread* our arrays of min and max temperatures so we get one flat array with both arrays concatenated. If you're unfamiliar with this syntax, feel free to read more here[a].
>
> [a] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax

We'll be converting a single data point into an `x` or `y` value many times – let's create two utility functions to help us to do just that. It seems simple enough now, but it's nice to have this logic in one place and not cluttering our `.attr()` functions.

code/11-radar-weather-chart/completed/chart.js

```
  const getXFromDataPoint = (d, offset=1.4) => getCoordinatesForAngle(
    angleScale(dateAccessor(d)),
    offset
  )[0]
  const getYFromDataPoint = (d, offset=1.4) => getCoordinatesForAngle(
    angleScale(dateAccessor(d)),
    offset
  )[1]
```

Let's put this scale to use! At the end of our **Draw peripherals** step, let's add a few circle grid lines that correspond to temperatures within our `radiusScale`.

code/11-radar-weather-chart/completed/chart.js

```
  const temperatureTicks = radiusScale.ticks(4)
  const gridCircles = temperatureTicks.map(d => (
    peripherals.append("circle")
      .attr("r", radiusScale(d))
      .attr("class", "grid-line")
  ))
```

Since `<circle>` elements default to a black fill, we won't be able to see much yet.

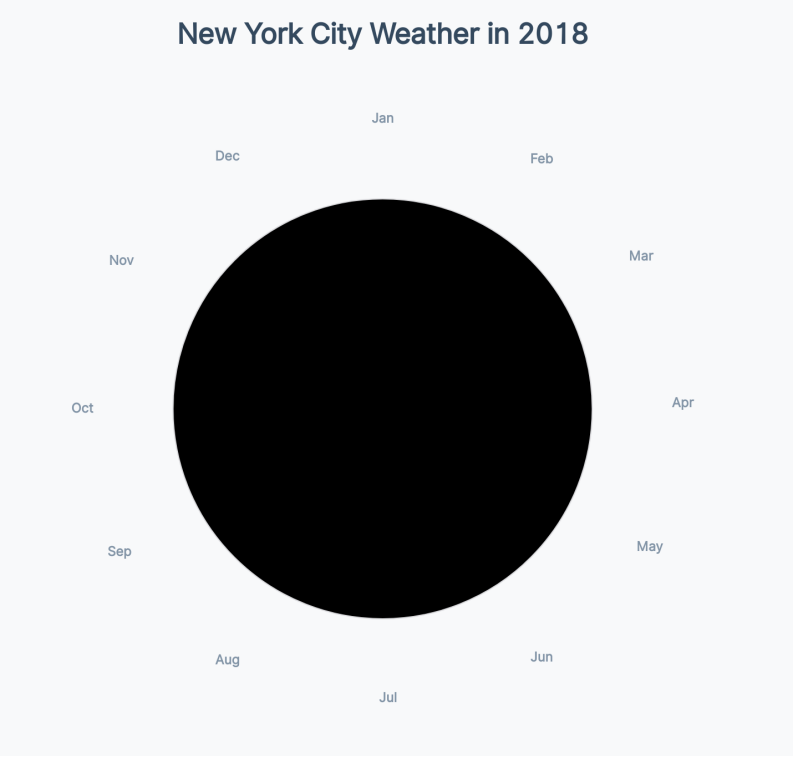


Chart with circles

Let's add some styles to remove the fill from any `.grid-line` elements and add a faint `stroke`.

```
.grid-line {
    fill: none;
    stroke: #dadadd;
}
```

Wonderful! Now we can see our concentric circles on our chart.

Chart with circles, styled

Similar to our month lines, we'll need labels to tell our viewers what temperature each of these circles represents.

code/11-radar-weather-chart/completed/chart.js

```
157   const tickLabels = temperatureTicks.map(d => {
158     if (!d) return
159     return peripherals.append("text")
160       .attr("x", 4)
161       .attr("y", -radiusScale(d) + 2)
162       .attr("class", "tick-label-temperature")
163       .html(`${d3.format(".0f")(d)}°F`)
164   })
```

> Notice that we're returning early if `d` is falsey – we don't want to make a label for a temperature of `0`.

We'll need to vertically center and dim our labels – let's update our `.tick-label-temperature` elements in our `styles.css` file.

```
.tick-label-temperature {
    fill: #8395a7;
    opacity: 0.7;
    font-size: 0.7em;
    dominant-baseline: middle;
}
```

These labels are very helpful, but they're a little hard to read on top of our grid lines.

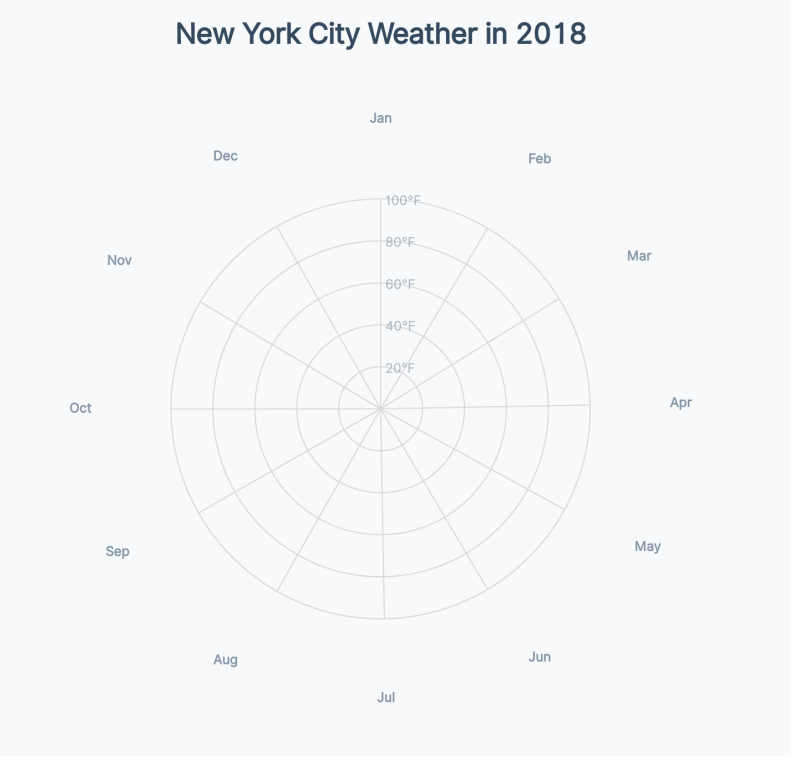


Chart with circles

Let's add a `<rect>` *behind* our labels that's the same color as the background of our page. We'll need to add this code before we draw our `tickLabels` and after our `gridCircles`, since SVG stacks elements in the order we draw them.

code/11-radar-weather-chart/completed/chart.js

```
149   const tickLabelBackgrounds = temperatureTicks.map(d => {
150     if (!d) return
151     return peripherals.append("rect")
152       .attr("y", -radiusScale(d) - 10)
153       .attr("width", 40)
154       .attr("height", 20)
155       .attr("fill", "#f8f9fa")
156   })
```

That's much easier to read!

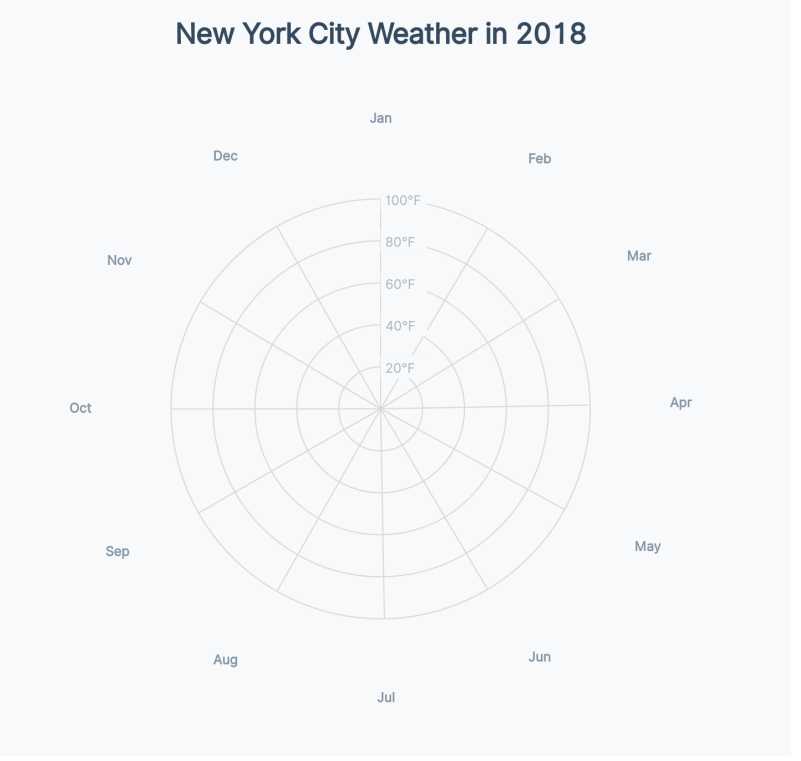


Chart with circles

Great! We're all set with our grid marks and ready to draw some data!

Adding freezing

Let's ease into drawing our data elements by drawing a `<circle>` to show where *freezing* is on our chart. We'll want to write this code in our **Draw data** step.

> As usual, either draw what "feels like" freezing to you or skip this step if your weather doesn't drop below freezing.

We can first check if our temperatures drop low enough:

code/11-radar-weather-chart/completed/chart.js

```
168   const containsFreezing = radiusScale.domain()[0] < 32
```

If our temperatures do drop below freezing, we'll add a `<circle>` whose radius ends at 32 degrees Fahrenheit.

```
if (containsFreezing) {
 const freezingCircle = bounds.append("circle")
   .attr("r", radiusScale(32))
   .attr("class", "freezing-circle")
}
```

Let's set the fill color and opacity of our circle to be a light cyan.

```
.freezing-circle {
    fill: #00d2d3;
    opacity: 0.15;
}
```

Great! Now we can see where the freezing temperatures will lie on our chart.

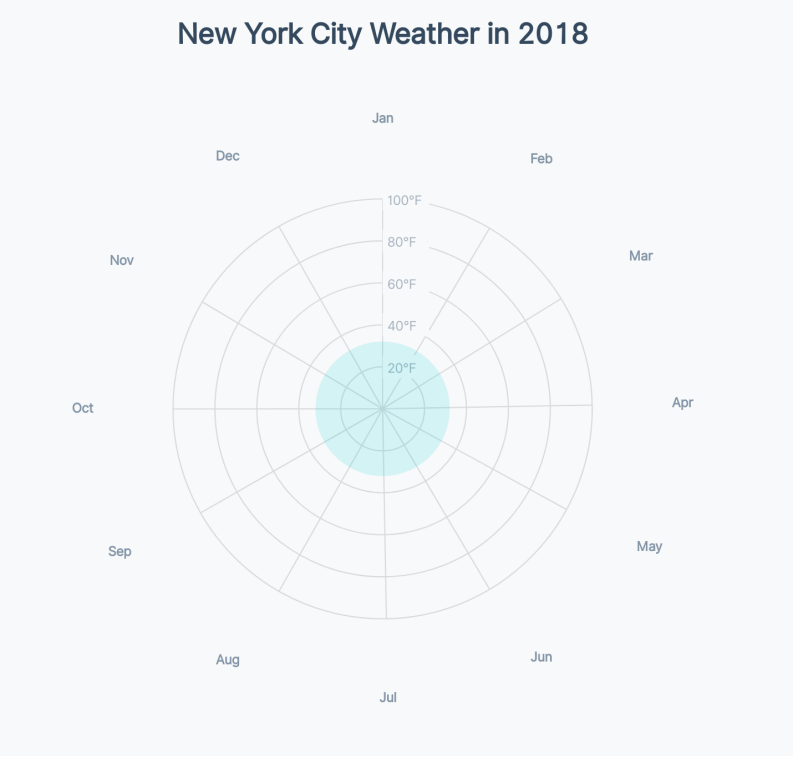


Chart with freezing point

Adding the temperature area

Our finished chart has a shape that covers the minimum and maximum temperatures for each day. Our first instinct to draw an area is to use `d3.area()`, but `d3.area()` will only take an `x` and a `y` position.

Instead, we want to use d3.areaRadial()[71], which is similar to `d3.area()`, but has `.angle()` and `.radius()` methods. Since we want our area to span the minimum and maximum temperature for a day, we can use `.innerRadius()` and `.outerRadius()` instead of one `.radius()`.

[71] https://github.com/d3/d3-shape#areaRadial

code/11-radar-weather-chart/completed/chart.js

```
175   const areaGenerator = d3.areaRadial()
176       .angle(d => angleScale(dateAccessor(d)))
177       .innerRadius(d => radiusScale(temperatureMinAccessor(d)))
178       .outerRadius(d => radiusScale(temperatureMaxAccessor(d)))
```

Like `.line()` and `.area()` generators, our `areaGenerator()` will return the `d` attribute string for a `<path>` element, given a dataset. Let's create a `<path>` element and set its `d` attribute.

```
const area = bounds.append("path")
   .attr("class", "area")
   .attr("d", areaGenerator(dataset))
```

Perfect! Now our chart is starting to take shape.